

MANIPAL ACADEMY OF HIGHER EDUCATION
SECOND SEMESTER B.TECH. DEGREE EXAMINATIONS - APRIL/MAY 2026
SUBJECT: CSS 1002/CSS_1002 - INTRODUCTION TO OBJECT ORIENTED
PROGRAMMING- SCHEME

Qn.No	Question	M														
1A.	Design a Java class Counter that contains a static variable totalObjects to keep track of the number of objects created. Increment this variable inside every constructor call. Also, implement a static method getTotalObjects() to return the current count. In the main method, create five objects of the Counter class and print the total number of objects after each instantiation. Further, answer the following: i) What happens if the static variable totalObjects is accessed using an object reference instead of the class name? Will it result in a compile-time error or a warning? Justify your answer. ii) Distinguish between static variables and instance variables in terms of memory allocation (stack, heap, method area) and their behavior. Solution and Scheme: <pre>class Counter { static int totalObjects = 0; Counter() { totalObjects++; } static int getTotalObjects() { return totalObjects; } public static void main(String[] args) { for (int i = 0; i < 5; i++) { new Counter(); System.out.println("Total objects: " + getTotalObjects()); } } } // Output: 1, 2, 3, 4, 5</pre> <table><tr><th colspan="2">Code (3 marks)</th></tr><tr><td>Correct class with static int totalObjects = 0 declared at class level.</td><td>1 m</td></tr><tr><td>Constructor increments totalObjects++; static method getTotalObjects() returns the count correctly.</td><td>1 m</td></tr><tr><td>main() creates five objects and prints total after each: output is 1, 2, 3, 4, 5 on successive lines.</td><td>1 m</td></tr><tr><th colspan="2">Theory (2 marks)</th></tr><tr><td>Part (i): Accessing a static variable via an object reference (e.g. obj.totalObjects) is a WARNING, not a compile-time error. The compiler resolves it as the class-level variable. Award mark for stating 'warning, not error' with justification.</td><td>1 m</td></tr><tr><td>Part (ii): Static variables — one copy, stored in the method area (metaspace), shared across all instances. Instance variables — one copy per object, stored in the heap. Local variables live on the stack. Accept tabular or descriptive comparison.</td><td>1 m</td></tr></table>	Code (3 marks)		Correct class with static int totalObjects = 0 declared at class level.	1 m	Constructor increments totalObjects++; static method getTotalObjects() returns the count correctly.	1 m	main() creates five objects and prints total after each: output is 1, 2, 3, 4, 5 on successive lines.	1 m	Theory (2 marks)		Part (i): Accessing a static variable via an object reference (e.g. obj.totalObjects) is a WARNING, not a compile-time error. The compiler resolves it as the class-level variable. Award mark for stating 'warning, not error' with justification.	1 m	Part (ii): Static variables — one copy, stored in the method area (metaspace), shared across all instances. Instance variables — one copy per object, stored in the heap. Local variables live on the stack. Accept tabular or descriptive comparison.	1 m	5
Code (3 marks)																
Correct class with static int totalObjects = 0 declared at class level.	1 m															
Constructor increments totalObjects++; static method getTotalObjects() returns the count correctly.	1 m															
main() creates five objects and prints total after each: output is 1, 2, 3, 4, 5 on successive lines.	1 m															
Theory (2 marks)																
Part (i): Accessing a static variable via an object reference (e.g. obj.totalObjects) is a WARNING, not a compile-time error. The compiler resolves it as the class-level variable. Award mark for stating 'warning, not error' with justification.	1 m															
Part (ii): Static variables — one copy, stored in the method area (metaspace), shared across all instances. Instance variables — one copy per object, stored in the heap. Local variables live on the stack. Accept tabular or descriptive comparison.	1 m															
1B.	Write a Java code snippet that reads an integer score (0-100) and prints a grade using both an if-else-if ladder and a switch statement (using integer division). - Grade rule: 90-100 → A, 80-89 → B, 70-79 → C, 60-69 → D, below 60 → F - For the switch version, use score / 10 as the switch expression and handle cases 10 and 9 together for grade A - State one advantage of switch over if-else-if for this specific problem	3														

	Solution and Scheme: <pre>// if-else-if ladder if (score >= 90) grade = 'A'; else if (score >= 80) grade = 'B'; else if (score >= 70) grade = 'C'; else if (score >= 60) grade = 'D'; else grade = 'F'; // switch version switch (score / 10) { case 10: case 9: System.out.println("A"); break; case 8: System.out.println("B"); break; case 7: System.out.println("C"); break; case 6: System.out.println("D"); break; default: System.out.println("F"); }</pre> <table> <tr> <th colspan="2">Grade using if-else-if ladder and switch statement</th><th>3 marks</th></tr> <tr> <td></td><td>Correct if-else if ladder covering all five grade ranges (A, B, C, D, F) with proper boundary conditions (>=90, >=80, >=70, >=60, else).</td><td>1 m</td></tr> <tr> <td></td><td>Switch on score/10: cases 10 & 9 grouped/fall-through -> A; 8 -> B; 7 -> C; 6 -> D; default -> F. Both cases 10 and 9 must be handled together.</td><td>1 m</td></tr> <tr> <td></td><td>Advantage: switch is more readable/efficient for discrete integer values — no repeated evaluation of the variable. Accept any valid point (jump table, clarity, no re-evaluation).</td><td>1 m</td></tr> </table>	Grade using if-else-if ladder and switch statement		3 marks		Correct if-else if ladder covering all five grade ranges (A, B, C, D, F) with proper boundary conditions (>=90, >=80, >=70, >=60, else).	1 m		Switch on score/10: cases 10 & 9 grouped/fall-through -> A; 8 -> B; 7 -> C; 6 -> D; default -> F. Both cases 10 and 9 must be handled together.	1 m		Advantage: switch is more readable/efficient for discrete integer values — no repeated evaluation of the variable. Accept any valid point (jump table, clarity, no re-evaluation).	1 m	
Grade using if-else-if ladder and switch statement		3 marks												
	Correct if-else if ladder covering all five grade ranges (A, B, C, D, F) with proper boundary conditions (>=90, >=80, >=70, >=60, else).	1 m												
	Switch on score/10: cases 10 & 9 grouped/fall-through -> A; 8 -> B; 7 -> C; 6 -> D; default -> F. Both cases 10 and 9 must be handled together.	1 m												
	Advantage: switch is more readable/efficient for discrete integer values — no repeated evaluation of the variable. Accept any valid point (jump table, clarity, no re-evaluation).	1 m												
1C.	What is encapsulation? How does it differ from data hiding? Write a small Java class Person with one private field and a public method to demonstrate encapsulation. Solution and Scheme: <pre>class Person { private String name; // data hiding public String getName() { return name; } public void setName(String n) { this.name = n; } // controlled access }</pre> <table> <tr> <th colspan="2">Encapsulation vs data hiding + Person class</th><th>2 marks</th></tr> <tr> <td></td><td>Encapsulation = bundling data (fields) and methods into a single unit (class). Data hiding = restricting direct access using private. Key distinction: encapsulation is the broader concept; data hiding is a mechanism to achieve it.</td><td>1 m</td></tr> <tr> <td></td><td>Person class with at least one private field (e.g. name) and a public getter/setter demonstrating controlled access. Field must be private and method must be public.</td><td>1 m</td></tr> </table>	Encapsulation vs data hiding + Person class		2 marks		Encapsulation = bundling data (fields) and methods into a single unit (class). Data hiding = restricting direct access using private. Key distinction: encapsulation is the broader concept; data hiding is a mechanism to achieve it.	1 m		Person class with at least one private field (e.g. name) and a public getter/setter demonstrating controlled access. Field must be private and method must be public.	1 m	2			
Encapsulation vs data hiding + Person class		2 marks												
	Encapsulation = bundling data (fields) and methods into a single unit (class). Data hiding = restricting direct access using private. Key distinction: encapsulation is the broader concept; data hiding is a mechanism to achieve it.	1 m												
	Person class with at least one private field (e.g. name) and a public getter/setter demonstrating controlled access. Field must be private and method must be public.	1 m												
2A.	Consider the following Java program: <pre>class Vector { int[] data; int size; Vector(int n) { this.size = n; this.data = new int[n]; } // Constructor B - copy constructor Vector(Vector other) { this.size = other.size; this.data = other.data; // Line X } void set(int i, int val) { data[i] = val; } int get(int i) { return data[i]; }</pre>	5												

```

}
public class Test {
public static void main(String[] args) {
Vector A = new Vector(3);
A.set(0, 5);
Vector B = new Vector(A); // Line Y
B.set(0, 99);
System.out.println(A.get(0)); // Line Z
}}

```

1. What does Line Z print? Explain the concept of aliasing in this context.
2. Rewrite Constructor B to perform a deep copy of the 1D array so that changes in one object do not affect the other.
3. Add an overloaded static method:
static Vector add(Vector A, Vector B)
that returns a new Vector representing element-wise addition of two vectors.
What should the method do if the sizes of the two vectors do not match?
Design a solution without using exceptions (use a null return).
Critically evaluate whether this is a good design choice.

Solution and Scheme:

```

// Deep copy constructor
Vector(Vector other) {
    this.size = other.size;
    this.data = new int[other.size];
    for (int i = 0; i < size; i++) this.data[i] = other.data[i];
}

// Static add method
static Vector add(Vector A, Vector B) {
    if (A.size != B.size) return null; // null return (poor design
    - see scheme)
    Vector result = new Vector(A.size);
    for (int i = 0; i < A.size; i++) result.data[i] = A.data[i] +
    B.data[i];
    return result;
}

```

Vector class: aliasing, deep copy, overloaded add()		5 marks
	Line Z output: 99. Constructor B copies the array reference (this.data = other.data), so A and B point to the same array. Setting B[0] = 99 also changes A[0]. Full mark for output + aliasing explanation.	1 m
	Deep copy: replace Line X with this.data = new int[other.size]; followed by System.arraycopy or a for-loop copying each element.	1 m
	Static add() method returns a new Vector with element-wise sums. Returns null if sizes differ. Award 1 m for correct addition logic + new Vector; 1 m for size check + null return.	2 m
	Critical evaluation: returning null is poor design — callers must check or risk NullPointerException. Exceptions or Optional are better practice. Award mark for any valid critique with justification.	1 m

- 2B.** Read the multi-file scenario below and answer all sub-parts. The code spans two packages -carefully track what compiles, what does not, and why.
- File 1 - package shapes
- ```

package shapes;
public abstract class Shape {
int sides; // package-private
public abstract double area();

```

```

public String describe() {
 return "I have " + sides + " sides";
}
}
File 2 - package app
package app;
import shapes.Shape;
public class Square extends Shape {
 double side;
 public Square(double side) {
 this.side = side;
 this.sides = 4; // Line P
 }
 public double area() { return side * side; }
}
class Test {
 public static void main(String[] args) {
 Square sq = new Square(5);
 System.out.println(sq.area()); // Line Q
 System.out.println(sq.describe()); // Line R
 System.out.println(sq.sides); // Line S
 }
}

```

Part (a): Line P accesses sides from a different package - does it compile? Justify using Java's access modifier rules for package-private fields in inherited classes

Part (b): Line S is inside the same file as Square but in package app - does sq.sides compile? Is access to inherited package-private fields the same as access to directly declared ones?

Part (c): Lines Q and R - state their output. describe() references sides internally - does it print correctly? Explain the memory layout: where does sides live in the Square object?

### **Solution and Scheme:**

The image displays four screenshots of an IDE, likely Eclipse, showing the package structure and code for the shapes and app packages.

**Top Screenshot:** Shows the Package Explorer with the shapes package containing Shape.java. The main editor shows Shape.java with the following code:

```

1 package shapes;
2
3 public abstract class Shape {
4 int sides; // package-private
5
6 public abstract double area();
7
8 public String describe() {
9 return "I have " + sides + " sides";
10 }
11
12 }

```

**Second Screenshot:** Shows the Package Explorer with the app package containing Square.java. The main editor shows Square.java with the following code:

```

1 package app;
2
3 import shapes.Shape;
4
5 public class Square extends Shape {
6 double side;
7
8 public Square(double side) {
9 this.side = side;
10 // this.sides = 4; // Line P
11 }
12
13 public double area() {
14 return side * side;
15 }
16 }

```

**Third Screenshot:** Shows the Package Explorer with the app package containing Test.java. The main editor shows Test.java with the following code:

```

1 package app;
2
3 import shapes.Shape;
4
5 class Test {
6 public static void main(String[] args) {
7 Square sq = new Square(5);
8 System.out.println(sq.area()); // Line Q
9 System.out.println(sq.describe()); // Line R
10 System.out.println(sq.sides); // Line S
11 }
12 }
13

```

**Bottom Screenshot:** Shows the Package Explorer with the app package containing Test.java. The main editor shows Test.java with the following code:

```

1 package app;
2
3 import shapes.Shape;
4
5 class Test {
6 public static void main(String[] args) {
7 Square sq = new Square(5);
8 System.out.println(sq.area()); // Line Q
9 System.out.println(sq.describe()); // Line R
10 // System.out.println(sq.sides); // Line S
11 }
12 }
13

```

|                                                                                                                                                                                                                                                                         | <table><tr><th colspan="2">Multi-package access modifier analysis</th><th>5 marks</th></tr><tr><td colspan="3">Part (a) — Line P: <i>this.sides = 4</i> in Square (different package)</td></tr><tr><td>Does NOT compile. sides is package-private (default access) in shapes.Shape. A subclass in a different package does not inherit package-private members — only public and protected members are inherited across packages.</td><td>1.5 m</td></tr><tr><td colspan="3">Part (b) — Line S: <i>sq.sides</i> in package app</td></tr><tr><td>Does NOT compile. Even though Test is in the same file as Square, it is in package app, not shapes. Package-private access is package-scoped, not file-scoped. Square did not inherit sides (Line P failed), so sq.sides also fails.</td><td>1.5 m</td></tr><tr><td colspan="3">Part (c) — Lines Q and R</td></tr><tr><td>Line Q (sq.area()): compiles and prints 25.0. Line R (sq.describe()): describe() is in shapes package so it can access sides (= 0 since Line P fails). Prints 'I have 0 sides'. sides lives in the heap within the Square object instance.</td><td>1 m</td></tr></table>              | Multi-package access modifier analysis          |  | 5 marks | Part (a) — Line P: <i>this.sides = 4</i> in Square (different package) |     |                                                                                                                                                                                                                                                                         | Does NOT compile. sides is package-private (default access) in shapes.Shape. A subclass in a different package does not inherit package-private members — only public and protected members are inherited across packages. | 1.5 m | Part (b) — Line S: <i>sq.sides</i> in package app |  |  | Does NOT compile. Even though Test is in the same file as Square, it is in package app, not shapes. Package-private access is package-scoped, not file-scoped. Square did not inherit sides (Line P failed), so sq.sides also fails. | 1.5 m | Part (c) — Lines Q and R |  |  | Line Q (sq.area()): compiles and prints 25.0. Line R (sq.describe()): describe() is in shapes package so it can access sides (= 0 since Line P fails). Prints 'I have 0 sides'. sides lives in the heap within the Square object instance. | 1 m |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------|--|---------|------------------------------------------------------------------------|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|---------------------------------------------------|--|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|--------------------------|--|--|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----|--|
| Multi-package access modifier analysis                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 5 marks                                         |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Part (a) — Line P: <i>this.sides = 4</i> in Square (different package)                                                                                                                                                                                                  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Does NOT compile. sides is package-private (default access) in shapes.Shape. A subclass in a different package does not inherit package-private members — only public and protected members are inherited across packages.                                              | 1.5 m                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Part (b) — Line S: <i>sq.sides</i> in package app                                                                                                                                                                                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Does NOT compile. Even though Test is in the same file as Square, it is in package app, not shapes. Package-private access is package-scoped, not file-scoped. Square did not inherit sides (Line P failed), so sq.sides also fails.                                    | 1.5 m                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Part (c) — Lines Q and R                                                                                                                                                                                                                                                |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Line Q (sq.area()): compiles and prints 25.0. Line R (sq.describe()): describe() is in shapes package so it can access sides (= 0 since Line P fails). Prints 'I have 0 sides'. sides lives in the heap within the Square object instance.                              | 1 m                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| 2C.                                                                                                                                                                                                                                                                     | <p>Predict the output of the following program. Identify the role of super() in the subclass constructor.</p> <pre>class Vehicle { String type; Vehicle(String t) { type = t; System.out.println("Vehicle: " + type); }} class Car extends Vehicle { int speed; Car(String t, int s) { super(t); speed = s; System.out.println("Car speed: " + speed); }} class Test { public static void main(String[] args) { Car c = new Car("Sedan", 120); }</pre> <p><b>Solution and Scheme:</b></p> <pre>// Output when new Car("Sedan", 120) is called: Vehicle: Sedan Car speed: 120 // super(t) calls Vehicle(String t) first, then Car body executes.</pre> <table><tr><th colspan="2">super() in constructor chaining — Vehicle / Car</th><th>2 marks</th></tr><tr><td>Predicted output (in order): Vehicle: Sedan Car speed: 120</td><td>1 m</td></tr><tr><td>Role of super(t): explicitly calls the parent class (Vehicle) constructor with argument t, initialising type before Car's constructor body runs. Without it, compiler inserts no-arg super() which would fail (Vehicle has no no-arg constructor) — compile-time error.</td><td>1 m</td></tr></table> | super() in constructor chaining — Vehicle / Car |  | 2 marks | Predicted output (in order): Vehicle: Sedan Car speed: 120             | 1 m | Role of super(t): explicitly calls the parent class (Vehicle) constructor with argument t, initialising type before Car's constructor body runs. Without it, compiler inserts no-arg super() which would fail (Vehicle has no no-arg constructor) — compile-time error. | 1 m                                                                                                                                                                                                                        | 2     |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| super() in constructor chaining — Vehicle / Car                                                                                                                                                                                                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | 2 marks                                         |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Predicted output (in order): Vehicle: Sedan Car speed: 120                                                                                                                                                                                                              | 1 m                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| Role of super(t): explicitly calls the parent class (Vehicle) constructor with argument t, initialising type before Car's constructor body runs. Without it, compiler inserts no-arg super() which would fail (Vehicle has no no-arg constructor) — compile-time error. | 1 m                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                 |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |
| 3A.                                                                                                                                                                                                                                                                     | <p>Define abstract Class Canine with private final String RFID_CODE and private String location variables. It Include a constructor to initialize both variables. Define abstract methods <i>makeNoise()</i> and <i>eat()</i>, and an override <i>toString()</i> method from Object class to print properties (2 marks)</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | 5                                               |  |         |                                                                        |     |                                                                                                                                                                                                                                                                         |                                                                                                                                                                                                                            |       |                                                   |  |  |                                                                                                                                                                                                                                      |       |                          |  |  |                                                                                                                                                                                                                                            |     |  |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |   |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
|     | <p>Define Concrete Class Wolf extends Canine with a constructor passing parameters to the superclass and overrides <i>makeNoise()</i> prints “Wolf Howls “ and <i>eat()</i> prints “Wolf eats meat”. (2 marks)</p> <p>Define Demo Class Animal Management System. In its main method, demonstrate Dynamic Method Dispatch by using an Canine reference variable pointing to a Wolf object. Invoke <i>makeNoise()</i>, <i>eat()</i>, and print the object. (1 mark)</p> <pre> Scheme: <b>abstract class</b> Canine {     <b>private final</b> String RFID_CODE;     <b>private</b> String location;     Canine(String rfidCode, String location) {         <b>this</b>.RFID_CODE = rfidCode;         <b>this</b>.location = location;     }     <b>abstract void</b> makeNoise();     <b>abstract void</b> eat();     @Override     <b>public</b> String toString() {         <b>return</b> "RFID_CODE: " + RFID_CODE + ", Location: " + location;     } }  <b>class</b> Wolf <b>extends</b> Canine {     Wolf(String rfidCode, String location) {         <b>super</b>(rfidCode, location);     }     @Override <b>void</b> makeNoise() {         System.out.println("Wolf Howls");     }     @Override <b>void</b> eat() {         System.out.println("Wolf eats meat");     } }  <b>public class</b> AnimalManagementSystem {     <b>public static void</b> main(String[] args) {         Canine canine = <b>new</b> Wolf("W123", "Bandipur North");         System.out.println(canine);         canine.makeNoise();         canine.eat();     } } </pre> |   |
| 3B. | <p>A smart traffic control system manages signal lights at an intersection. The behavior of each signal (Red, Green, Yellow) varies dynamically based on traffic conditions.</p> <p><b>Task:</b></p> <ol style="list-style-type: none"> <li>Define an interface Signal with a method display().</li> <li>Implement this interface in three classes: RedSignal, GreenSignal, and YellowSignal, each displaying appropriate messages.</li> <li>Demonstrate run-time polymorphism in main.</li> </ol> <p>Scheme:</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | 3 |

|            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |          |
|------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|
|            | <pre> // Interface interface Signal {     void display(); }  // Red Signal class RedSignal implements Signal {     public void display() {         System.out.println("RED Signal: STOP");     } }  // Green Signal class GreenSignal implements Signal {     public void display() {         System.out.println("GREEN Signal: GO");     } }  // Yellow Signal class YellowSignal implements Signal {     public void display() {         System.out.println("YELLOW Signal: WAIT");     } }  // Main Class class TrafficTest {     public static void main(String[] args) {         Signal s;         s = new RedSignal();         s.display();          s = new GreenSignal();         s.display();          s = new YellowSignal();         s.display();     } } </pre> <p>Interface Defn – 1M<br/> Implementation by 3 classes – 1M<br/> Demonstration of run-time polymorphism in main – 1M</p> |          |
| <b>3C.</b> | <p>design a Java program that creates a package mathpack containing a class Add with methods to perform addition. Demonstrate how this class can be accessed from another class, and explain the interaction between the package and the accessing classes. (2 M)</p> <p>Ans)</p> <pre> // File: Add.java package mathpack; public class Add {     public int sum(int a, int b) { </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <b>2</b> |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |   |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
|     | <pre>         return a + b;     } }  (1 M) // File: Test.java import mathpack.Add; class Test {     public static void main(String[] args) {         Add obj = new Add();         System.out.println(obj.sum(2, 3));     } }  (1 M) Total- 2 Marks </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |   |
| 4A. | <p>A. Develop a Java program for a Smart Appliance Control System where different appliances can be controlled using a common interface.</p> <ol style="list-style-type: none"> <li>Create an interface ApplianceControl with the method operateDevice().</li> <li>Create two classes SmartLight and SmartFan that implement the ApplianceControl interface. The SmartLight class should include attributes such as brightnessLevel, while the SmartFan class should include speedLevel.</li> <li>Define the constructors to initialize the data.</li> <li>In the main() method, create an interface reference of type ApplianceControl and use it to control both devices by assigning objects of SmartLight and SmartFan. Display appropriate messages when the devices operate.</li> </ol> <p>(3 Marks)</p> <p>B. What is the output of the following code? Justify your answer.</p> <p>(2 Marks)</p> <p>Scheme:</p> <p>Evaluation Scheme:</p> <p>Class SmartLight : 1 Mark</p> <p>Class SmartFan- 1mark</p> <p>Main Method and Interface: 1 Mark</p> <p>B:</p> <p>Correct Output : 1 Mark</p> <p>Justification : 1 Mark</p> <p><b>Solution:</b></p> <pre> package endsem; interface ApplianceControl {     void operateDevice();} class SmartLight implements ApplianceControl {     int brightnessLevel;     SmartLight(int brightnessLevel) {         this.brightnessLevel = brightnessLevel; }     public void operateDevice() {         System.out.println("Smart Light is operating.");         System.out.println("Brightness Level set to: " + brightnessLevel);     } } class SmartFan implements ApplianceControl { </pre> | 5 |



|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |   |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
|     | <pre> <b>int</b> speedLevel; SmartFan(<b>int</b> speedLevel) {     <b>this</b>.speedLevel = speedLevel; } <b>public void</b> operateDevice() {     System.<i>out</i>.println("Smart Fan is operating.");     System.<i>out</i>.println("Speed Level set to: " + speedLevel); } } <b>public class</b> s1_q4A { <b>public static void</b> main(String[] args) {     ApplianceControl device;     device = <b>new</b> SmartLight(5);     device.operateDevice();     System.<i>out</i>.println();     device = <b>new</b> SmartFan(3);     device.operateDevice(); } } </pre> <p>Output: 25<br/>15</p> <p>Justification: Class C implements both the interfaces. Hence, methods of both the interfaces are implemented in class properly using public keyword.</p>                                                                                                                                                                                             |   |
| 4B. | <p>What are abstract methods and variables in an interface? Create an interface <b>Volume</b> containing a method to <b>calculateVolume</b>. Using Java, implement this interface in Cube and Cylinder classes and display the results.</p> <p><b>Scheme:</b></p> <p><b>Abstract Methods:</b> Methods declared in an interface do not have a body. They are implicitly abstract and must be implemented by the class that implements the interface. (0.5)</p> <p><b>Variables in Interface:</b> All variables declared in an interface are implicitly public static final<br/>This means they behave like constants and cannot be changed. (0.5)</p> <p><b>Java Program:</b></p> <pre> interface Volume {     double calculateVolume(); } (0.5)  // Cube class (0.5) class Cube implements Volume {     double side;      Cube(double s) {         side = s;     }      public double calculateVolume() {         return side * side * side;     } } </pre> | 3 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |   |
|-----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
|     | <pre> // Cylinder class (0.5)  class Cylinder implements Volume {     double radius, height;      Cylinder(double r, double h) {         radius = r;         height = h;     }      public double calculateVolume() {         return Math.PI * radius * radius * height;     } }  // Main class (0.5) class Demo {     public static void main(String[] args) {          Volume v1 = new Cube(3);         Volume v2 = new Cylinder(2, 5);          System.out.println("Volume of Cube: " + v1.calculateVolume());         System.out.println("Volume of Cylinder: " + v2.calculateVolume());     } } </pre>                                                                                                                                                                 |   |
| 4C. | <p>Write a Java program to demonstrate the use of <b>static import</b>.</p> <ul style="list-style-type: none"> <li>Create a package utility with a class Calculator containing static methods:<br/>add(int, int) and square(int).</li> <li>In another class:             <ul style="list-style-type: none"> <li>Use <b>import</b> statement to access these methods</li> <li>Invoke the methods</li> <li>Display the results</li> </ul> </li> </ul> <p><b>Scheme: 1M each class</b><br/>package utility;</p> <pre> public class Calculator {      // Static method to add two numbers     public static int add(int a, int b) {         return a + b;     }      // Static method to find square     public static int square(int a) {         return a * a;     } } </pre> | 2 |

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |   |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
|     | <pre>     } } import static utility.Calculator.add; import static utility.Calculator.square;  public class MainApp {      public static void main(String[] args) {  int sum = add(10, 20);     int sq = square(5);      System.out.println("Sum = " + sum);     System.out.println("Square = " + sq);     } } </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |   |
| 5A. | <p>Create a class <b>Product</b> with two data members <b>id(int)</b> and <b>review(String)</b>. Provide a parameterized constructor to initialize the data members and a <b>display()</b> method to output the product details. Create the driver class <b>ProductDemo</b> to initialize an array of products whose details are taken from the user. If the string entered for review does not include the phrases- “Good/good” or “Bad/bad” or “Satisfactory/satisfactory”, then raise <b>InvalidReviewException</b> (user-defined) with an appropriate message.</p> <p style="text-align: right;">3M</p> <p>Illustrate exception handling mechanism with a specific example in java</p> <p style="text-align: right;">2M</p> <p>Scheme:<br/> <b>1M for each class.</b></p> <pre> import java.util.Scanner; class InvalidReviewException extends Exception{ String r; InvalidReviewException(String r){ this.r = r; } public String toString(){ return "Invalid Review:"+r; } } class Product{ int id; String review; Product(int i, String r) throws InvalidReviewException{ id=i; review = r; if(review.indexOf("Good") ==-1 &amp;&amp; review.indexOf("good")==-1 &amp;&amp; review.indexOf("Bad") ==-1 &amp;&amp; review.indexOf("bad")==-1 &amp;&amp; review.indexOf("Satisfactory") ==-1 &amp;&amp; review.indexOf("satisfactory")==-1) </pre> <p style="text-align: right;">1M</p> | 5 |

```

throw new InvalidReviewException(r);
}
void display(){
System.out.println("Id:"+id+"Review:"+review);
}
}
1M
class ProductReviewDemo{
public static void main(String[] args){
Scanner sc = new Scanner(System.in);
Product[] p = new Product[3];
for(int i=0; i<p.length; i++){
try {
System.out.println("Id:");
int id = sc.nextInt();
sc.nextLine();
System.out.println("Review:");
String r = sc.nextLine();
p[i] = new Product(id,r);
}
catch(InvalidReviewException ie){
System.out.println(ie);
}
}
}
}
1M

```

- A Java exception is an object that describes an exceptional (that is, error) condition that has occurred in a piece of code
- When an exceptional condition arises, an object representing that exception is created and *thrown* in the method that caused the error
- That method may choose to
  - a) handle itself
  - or
  - b) pass it on.
- At some point the exception is *caught* and processed
- It is mainly done using **try, catch, and finally** blocks.

1M

### Example 1M

```

public class ExceptionDemo {
 public static void main(String[] args) {
 try {
 int a = 10;
 int b = 0;
 int c = a / b; // causes ArithmeticException
 System.out.println("Result: " + c);
 }
 catch (ArithmeticException e) {
 System.out.println("Exception caught: Division by zero is not
allowed.");
 }
 System.out.println("Program continues after exception handling.");
 }
}

```

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |   |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|
| 5B. | <p>Analyze how exception handling is performed in nested try blocks when an exception is thrown in the inner try block. Examine the flow of control and justify how the appropriate catch block is selected, supporting your explanation with a suitable example.</p> <p>Scheme:<br/>When <b>nested try blocks</b> are used, an exception thrown in the <b>inner try block</b> is handled as follows:</p> <ol style="list-style-type: none"> <li>1. Java first checks for a matching catch block <b>inside the inner try</b>.</li> <li>2. If a suitable catch is found → the exception is handled there.</li> <li>3. If not → the exception <b>propagates to the outer try block</b>.</li> <li>4. The outer catch block then attempts to handle it.</li> <li>5. If no matching handler exists at any level → the program terminates abnormally.</li> </ol> <p style="text-align: right;">- 1 Mark-</p> <pre> class NestedTryDemo {     public static void main(String[] args) {         try {             System.out.println("Outer try block");              try {                 System.out.println("Inner try block");                 int result = 10 / 0; // Exception occurs here             } catch (NullPointerException e) {                 System.out.println("Inner catch: NullPointerException handled");             }              System.out.println("After inner try-catch");          } catch (ArithmeticException e) {             System.out.println("Outer catch: ArithmeticException handled");         }         System.out.println("Program continues...");     } } </pre> <p style="text-align: right;">-1 Mark-</p> <p>Output:<br/>Outer try block<br/>Inner try block<br/>Outer catch: ArithmeticException handled<br/>Program continues...</p> <p style="text-align: right;">-0.5 Mark-</p> <p><b>Explanation (Flow Analysis)</b></p> <ul style="list-style-type: none"> <li>• The inner try block throws an <b>ArithmeticException</b> (10 / 0).</li> <li>• The inner catch block handles only <b>NullPointerException</b>, so: No match → exception is <b>not handled here</b></li> <li>• The exception propagates to the <b>outer try block</b>.</li> <li>• The outer catch block handles <b>ArithmeticException</b>: Exception handled successfully</li> <li>• Program continues normally after handling.</li> </ul> <p style="text-align: right;">- 0.5 Mark –</p> | 3 |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---|

**5C.**

Differentiate between throw and throws with syntax and examples.

**2**

| <b>throw</b>                                                                                                                                                                                                                 | <b>throws</b>                                                                                                                                                                                                     |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>-Used to explicitly throw an exception</li><li>- Transfers control to nearest catch block</li><li>- Only one exception object can be thrown at a time</li></ul>                        | <ul style="list-style-type: none"><li>-Used to declare exceptions that a method may throw</li><li>-Passes responsibility to caller method</li><li>-Can declare multiple exceptions</li></ul>                      |
| <p>Used inside method/block.<br/>example<br/>void method()<br/>{<br/>  try{<br/>    if (condition)<br/>      throw e;<br/>    .....<br/>  } catch(ExceptionType1 e){ }<br/>}</p> <p>Where e is<br/>some exception object</p> | <p>In method declaration (signature)<br/>void method() throws ExceptionType1,<br/>ExceptionType2<br/>{<br/>  if (condition1)<br/>    throw e1;<br/>  else if (condition2)<br/>    throw e2;<br/>  .....<br/>}</p> |